



Decoratori applicati

Python

Un decoratore in Python è una funzione speciale che permette di modificare o estendere il comportamento di un'altra funzione senza cambiarne direttamente il codice originale.

Il decoratore “avvolge” una funzione aggiungendo operazioni prima e/o dopo la sua esecuzione.



In pratica viene usato per:

- logging
- controlli di accesso
- misurazione del tempo
- validazione dati
- caching
- debug

La sintassi usa il simbolo @.



Il wrapper è la funzione interna creata dal decoratore.

Ha il compito di “avvolgere” la funzione originale aggiungendo nuovo comportamento prima, dopo o attorno alla sua esecuzione.

Quando un decoratore viene applicato:

- **la funzione originale non viene eseguita direttamente;**
- **viene sostituita dalla funzione wrapper;**
- **il wrapper decide cosa fare con la funzione originale.**



In pratica il flusso diventa:

Chiamata funzione → wrapper → funzione originale

Il wrapper viene usato perché permette di:

- aggiungere controlli;
- modificare input e output;
- eseguire codice automatico;
- mantenere separata la logica aggiuntiva dalla funzione principale.



```
1.# Decoratore
2.def saluto(funzione):
3.
4.    # Funzione interna
5.    def wrapper():
6.
7.        print("Prima della funzione")
8.
9.        # Esegue la funzione originale
10.       funzione()
11.
12.       print("Dopo la funzione")
13.
14.    return wrapper
15.
16.
17.# Applico il decoratore
18.@saluto
19.def messaggio():
20.    print("Ciao Mirko!")
21.
22.
23.# Chiamata funzione
24.messaggio()
```

Spiegazione:

- **saluto()** è il decoratore
- **wrapper()** contiene il codice aggiuntivo
- **funzione()** rappresenta la funzione originale
- **@saluto** applica automaticamente il decoratore



```
1.# Variabile che simula il login
2.utente_loggato = True
3.
4.
5.# Decoratore
6.def richiede_login(funzione):
7.
8.    def wrapper():
9.
10.        if utente_loggato == True:
11.            funzione()
12.        else:
13.            print("Accesso negato")
14.
15.    return wrapper
16.
17.
18.# Funzione protetta
19.@richiede_login
20.def area_personale():
21.    print("Benvenuto nell'area personale")
22.
23.
24.# Avvio
25.area_personale()
```



```
1.import time
2.
3.
4.# Decoratore
5.def calcola_tempo(funzione):
6.
7.    def wrapper():
8.
9.        inizio = time.time()
10.
11.        funzione()
12.
13.        fine = time.time()
14.
15.        print("Tempo esecuzione:", fine - inizio)
16.
17.    return wrapper
18.
19.
20.@calcola_tempo
21.def operazione():
22.
23.    for i in range(1000000):
24.        pass
25.
26.
27.operazione()
```

Questo decoratore:

- salva il tempo iniziale
- esegue la funzione
- salva il tempo finale
- calcola la durata



I parametri nei decoratori servono per rendere il wrapper compatibile con le funzioni decorate.

Se una funzione originale riceve valori in ingresso, anche il wrapper deve poterli ricevere e passare alla funzione originale. Per questo motivo il wrapper spesso contiene parametri.

Esempio : `wrapper(nome, età)` ovvero args

Il wrapper:

1. riceve i parametri;
2. può controllarli o modificarli;
3. li passa alla funzione originale.

 Senza parametri il decoratore funzionerebbe solo con funzioni “vuote”, cioè senza input.

Nella pratica si usano spesso:

- `*args` → raccoglie parametri posizionali;
- `**kwargs` → raccoglie parametri con nome.

Questo permette al decoratore di funzionare con quasi qualsiasi funzione, indipendentemente dal numero di parametri.

Esempio: `wrapper(*args, **kwargs)`

significa:

- accetta qualsiasi parametro;
- inoltra tutto alla funzione originale.



È una tecnica molto usata perché rende i decoratori riutilizzabili e generici.

```
1.# Decoratore
2.def controllo(funzione):
3.
4.    # *args raccoglie tutti i parametri posizionali
5.    def wrapper(*args):
6.
7.        print("Avvio funzione")
8.
9.        # Passa tutti i parametri alla funzione originale
10.       funzione(*args)
11.
12.       print("Fine funzione")
13.
14.    return wrapper
15.
16.
17.# Funzione decorata
18.@controllo
19.def somma(a, b, c):
20.
21.    risultato = a + b + c
22.
23.    print("Risultato:", risultato)
24.
25.
26.# Chiamata funzione
27.somma(5, 10, 15)
```

Spiegazione

- ***args** raccoglie i valori 5, 10, 15
- vengono salvati come una tupla
- **funzione(*args)** reinvia automaticamente tutti i valori
- il decoratore funziona anche se il numero di parametri cambia



```
1.# Decoratore
2.def log(funzione):
3.
4.  # **kwargs raccoglie parametri con nome
5.  def wrapper(**kwargs):
6.
7.      print("Dati ricevuti:")
8.      print(kwargs)
9.
10.     # Passa i parametri alla funzione originale
11.     funzione(**kwargs)
12.
13.  return wrapper
14.
15.
16.# Funzione decorata
17.@log
18.def utente(nome, età):
19.
20.     print("Nome:", nome)
21.     print("Età:", età)
22.
23.
24.# Chiamata funzione
25.utente(nome="Mirko", età=30)
```

Spiegazione:

- ****kwargs raccoglie:**
 1. {
 2. "nome": "Mirko",
 3. "età": 30
 4. }
- i dati vengono salvati in un dizionario
- **funzione(**kwargs)** rimanda automaticamente i parametri originali
- utile quando non si conoscono in anticipo i nomi dei parametri



Errori comuni nei decoratori in Python

Uno degli errori più frequenti è dimenticare di restituire il wrapper; il decoratore deve sempre restituire la funzione interna che sostituisce quella originale. Se manca il return, la funzione decorata smette di funzionare correttamente.

Un altro errore comune è non eseguire la funzione originale all'interno del wrapper. In questo caso il decoratore aggiunge codice, ma la funzione vera e propria non viene mai chiamata.

Molti problemi nascono anche dalla gestione dei parametri.

Se la funzione decorata richiede parametri ma il wrapper non li accetta, Python genera errori di tipo, per questo spesso si usano `*args` e `**kwargs`, che permettono di ricevere qualsiasi combinazione di parametri.



Un altro errore frequente riguarda il valore di ritorno.

Se la funzione originale restituisce un risultato ma il wrapper non lo ritorna a sua volta, il valore viene perso, questo può creare bug difficili da individuare.

Dal punto di vista progettuale, un errore comune è rendere i decoratori troppo invasivi.; un decoratore dovrebbe aggiungere comportamento senza stravolgere la logica della funzione originale. Se modifica troppo il flusso del programma, il codice diventa difficile da leggere e mantenere.

Quando si usano molti decoratori insieme, il codice può diventare poco chiaro, più decoratori significano più livelli di esecuzione, e questo rende più difficile capire da dove provengono errori o modifiche al comportamento.



```
1. # Primo decoratore
2. def decoratore1(funzione):
3.
4.     def wrapper():
5.
6.         print("Prima esecuzione")
7.
8.         funzione()
9.
10.        print("Fine primo decoratore")
11.
12.    return wrapper
13.
14.
15. # Secondo decoratore
16. def decoratore2(funzione):
17.
18.     def wrapper():
19.
20.         print("Seconda esecuzione")
21.
22.         funzione()
23.
24.         print("Fine secondo decoratore")
25.
26.    return wrapper
27.
28.
29. # Funzione decorata con due decoratori
30. @decoratore1
31. @decoratore2
32. def messaggio():
33.
34.     print("Funzione originale")
35.
36.
37. # Avvio funzione
38. messaggio()
```

Spiegazione:

- @decoratore2 viene applicato per primo
- @decoratore1 avvolge il risultato del secondo decoratore

l'ordine reale è:

decoratore1(decoratore2(messaggio))

Ogni wrapper aggiunge comportamento prima e dopo la funzione originale, creando più livelli di esecuzione.



Buon Davante a tutti

